# 📘 Computer Science (Python) Fundamentals: Character Set, Tokens, Operators, Expressions & I/O

## 1. Python Character Set

**Definition:** A set of valid characters recognized by Python interpreter. Python uses **Unicode** character set. UTF-8 (Unicode Transformation Format) is the most widely used character set.

- **Letters:** A–Z, a–z
- **Digits:** 0–9
- **Special symbols:** `+ - * / % = < > ( ) [ ] { } . , ; : ! @ # $ & ~ ^ | \ ' " ` _`
- **Whitespace :** space, tab ( `\t` ), newline ( `\n` ), carriage return ( `\r` )
- **Case-sensitive:** `NAME` and `name` are different identifiers.
- **Unicode support :** Python supports international characters like ₹, ç, α, etc.

> ✓ Example: Any python program like `print("हिंदी")` uses unicode characters.

## 2. Python Tokens

**Definition:** Smallest individual unit in a Python program. They are the fundamental building blocks of a program. Classified into 5 types:
1. Keywords
2. Identifiers
3. Literals
4. Operators
5. Punctuators

### 2.1 Keywords

- Reserved words with special/fixed meaning.
- Cannot be used as variable names.
- 35 Keywords in Python:

| | | | | |
|---|---|---|---|---|
| `False` | `await` | `else` | `in` | `raise` |

| | | | | |
|---|---|---|---|---|
| `None` | `break` | `except` | `is` | `return` |
| `True` | `class` | `finally` | `lambda` | `try` |
| `and` | `continue` | `for` | `nonlocal` | `while` |
| `as` | `def` | `from` | `not` | `with` |
| `assert` | `del` | `global` | `or` | `yield` |
| `async` | `elif` | `if` | `pass` | `import` |

## 2.2 Identifiers

- Names (user-defined names) given to variables, functions, classes, objects, etc.
- **Rules:**
  - Should not contain special characters `(except _)`.
  - Should not contain blank spaces/white spaces.
  - Should not START with a digit.
  - Should not be a Python keyword.

```
Valid: _roll, student1, Mark, totalMarks

Invalid: 1stVar, my-name, total marks, for
```

## 2.3 Literals (Constants)

- String literal: `"Hello"` , `'CS'`
- Numeric literal: `25, 3.1415, 0b1010`
- Boolean literal: `True, False`
- Special literal: `None`

## 2.4 Operators

Symbols that perform operations:
1. Arithmetic
2. Relational
3. Logical
4. Membership
5. Identity

6. Assignment
7. Arithmetic/augmented assignment.

### 2.5 Punctuators

Symbols used for structuring code: `( )` `{ }` `[ ]` `,` `.` `:` `;` `@` `=` (assignment context)

## 3. Variables in Python

**Definition:** Named memory location that holds a value. Python is dynamically typed.

**Syntax:** `variable_name = value`

```
age = 17            # integer
name = "Riya"       # string
percentage = 92.6   # float
```

## 4. Concept of L-value and R-value

- **L-value (Location value)** → Refers to memory address; must appear on the left side of assignment operator. Example: `x = 10` → `x` is L-value.
- **R-value (Right value)** → Actual data or expression result that appears on right side. Example: `y = a + 5` → `a+5` is R-value.

```
a = 25      # a (L-value), 25 (R-value)

result = a + 10   # result is L-value, a+10 is R-value
```

## 5. Operators in Python

### 5.1 Arithmetic Operators

| Operator | Meaning | Example | Result |
|---|---|---|---|
| + | Addition | 7+3 | 10 |
| - | Subtraction | 10-4 | 6 |

| Operator | Meaning | Example | Result |
|---|---|---|---|
| * | Multiplication | 5*6 | 30 |
| / | Float division (regular) | 7/2 | 3.5 |
| // | Integer division (floor) | 7//2 | 3 |
| % | Modulus (remainder) | 17%5 | 2 |
| ** | Exponentiation | 2**5 | 32 |

## 5.2 Relational (Comparison) Operators → returns bool (True/False)

- `==` → equal to
- `!=` → not equal
- `>` → greater than
- `<` → less than
- `>=` → greater than or equal to
- `<=` → less than or equal to

```
x = 15
print(x > 10)    # True
print(x != 5)    # True
print(x <= 15)   # True
```

## 5.3 Logical Operators

- Used to check multiple conditions
- `and` → True if both operands are True
- `or` → True if at least one is True
- Used with one condition
- `not` → reverses the logical state

```
print((5>3) and (2<4))   # True

print(not (10==10))       # False
```

## 5.4 Assignment Operator

`=` assigns R-value to L-value. e.g., `marks = 95`

## 5.5 Augmented Assignment Operators

| Operator | Example | Equivalent to |
|---|---|---|
| += | x += 5 | x = x + 5 |
| -= | x -= 3 | x = x - 3 |
| *= | x *= 2 | x = x * 2 |
| /= | x /= 4 | x = x / 4 |
| //= | x //= 2 | x = x // 2 |
| %= | x %= 3 | x = x % 3 |
| **= | x **= 2 | x = x ** 2 |

## 5.6 Identity Operators ( `is` , `is not` )

Check if two variables refer to the same memory object.

```
a = [1,2]
b = a
c = [1,2]
print(a is b)      # True
print(a is c)      # False (different objects)
print(a is not c)  # True
```

## 5.7 Membership Operators ( `in` , `not in` )

Checks presence of value in sequence (string, list, tuple, dict).

```
fruits = ["apple", "banana"]
print("apple" in fruits)       # True
print("grapes" not in fruits)  # True
```

## 6. Expressions

**Definition:** Combination of variables, literals, operators and function calls that evaluates to a single value.

```
(a+b)*c - 4
age >= 18 and has_permission
```

## 7. Precedence of Operators (Highest to Lowest)

| Precedence | Operators | Associativity |
|---|---|---|
| 1 (highest) | () parentheses | Left to right |
| 2 | ** (exponent) | Right to left |
| 3 | +x, -x (unary), not | Right to left |
| 4 | *, /, //, % | Left to right |
| 5 | +, - (binary) | Left to right |
| 6 | ==, !=, >, <, >=, <=, is, is not, in, not in | Left to right |
| 7 | and | Left to right |
| 8 (lowest) | or | Left to right |

```
result = 10 + 3 * 2 ** 2
# 10 + 3 * 4 = 10+12 = 22
```

## 8. Evaluation of an Expression

Steps: precedence → associativity → substitute values → compute stepwise.

✓ Same precedence: left-to-right except for ** (right-to-left).
Use parentheses to override: `(5+3)*2` → 16

Python evaluates an expression in the following order:

1. Apply **operator precedence**.
2. If operators have the same precedence, use **associativity**.

3. Replace variables with their values.
4. Evaluate the expression step by step.

**Remember:**

- Operators with higher precedence are evaluated first.
- If operators have the same precedence, evaluation is from **left to right**.
- The exponent operator ( `**` ) is evaluated from **right to left**.

## Example 1: Operator Precedence

```
x = 5
y = 2

result = x + y * 3
print(result)
```

**Output**

```
11
```

**Step-by-Step Evaluation**

```
x + y * 3
= 5 + 2 * 3
= 5 + 6
= 11
```

---

## Example 2: Parentheses have Highest Priority

```
x = 5
y = 2

result = (x + y) * 3
print(result)
```

**Output**

```
21
```

**Step-by-Step Evaluation**

```
(x + y) * 3
= (5 + 2) * 3
= 7 * 3
= 21
```

---

## Example 3: Same Precedence (Left to Right)

```
result = 20 / 5 * 2
print(result)
```

**Output**

```
8.0
```

**Step-by-Step Evaluation**

```
20 / 5 * 2
= 4.0 * 2
= 8.0
```

Both `/` and `*` have the same precedence, so they are evaluated from **left to right**.

---

## Example 4: Exponent Operator (Right to Left)

```
result = 2 ** 3 ** 2
print(result)
```

**Output**

```
512
```

**Step-by-Step Evaluation**

```
2 ** 3 ** 2
= 2 ** (3 ** 2)
= 2 ** 9
= 512
```

The exponent operator ( `**` ) is evaluated from **right to left**.

---

## Example 5: Mixed Operators

```
a = 10
b = 5
c = 2

result = a + b * c - 4 / 2
print(result)
```

**Output**

```
18.0
```

**Step-by-Step Evaluation**

```
a + b * c - 4 / 2
= 10 + 5 * 2 - 4 / 2
= 10 + 10 - 2.0
= 20 - 2.0
= 18.0
```

---

## Example 6: Variables and Parentheses

```
x = 6
y = 4
z = 2

result = (x + y) / z
print(result)
```

**Output**

```
5.0
```

**Step-by-Step Evaluation**

```
(x + y) / z
= (6 + 4) / 2
= 10 / 2
= 5.0
```

### Example 7: Expression with Multiple Operators

```
result = 20 - 2 ** 3 ** 2 / 8 * 2 + 5
print(result)
```

**Output**

```
-103.0
```

**Step-by-Step Evaluation**

```
20 - 2 ** 3 ** 2 / 8 * 2 + 5
= 20 - 2 ** (3 ** 2) / 8 * 2 + 5
= 20 - 2 ** 9 / 8 * 2 + 5
= 20 - 512 / 8 * 2 + 5
= 20 - 64.0 * 2 + 5
= 20 - 128.0 + 5
= -108.0 + 5
= -103.0
```

**Explanation:**

1. `**` has the highest precedence and is evaluated from **right to left**.
2. `/` and `*` have the same precedence and are evaluated from **left to right**.
3. Finally, `+` and `-` are evaluated from **left to right**.

## Evaluation of Logical Expressions

When an expression contains logical operators ( `not` , `and` , `or` ), Python evaluates it according to the following precedence.

### Logical Operator Precedence

| Priority | Operator | Description |
|---|---|---|
| 1 (Highest) | `not` | Logical NOT |
| 2 | `and` | Logical AND |
| 3 (Lowest) | `or` | Logical OR |

**Remember:**

1. `not` is evaluated first.

2. `and` is evaluated next.
3. `or` is evaluated last.
4. Use parentheses `()` to change the order of evaluation.

---

## Example 1

```
print(True or False and False)
```

**Output**

```
True
```

**Evaluation**

```
True or False and False
= True or False
= True
```

---

## Example 2

```
print(not True or False)
```

**Output**

```
False
```

**Evaluation**

```
not True or False
= False or False
= False
```

---

## Example 3

```
print(not False and True or False)
```

**Output**

```
True
```

**Evaluation**

```
not False and True or False
= True and True or False
= True or False
= True
```

---

## Example 4

```
a = 10
b = 5

print(a > b and b < 10 or a == 0)
```

**Output**

```
True
```

**Evaluation**

```
a > b and b < 10 or a == 0
= True and True or False
= True or False
= True
```

---

## Example 5 (Using Parentheses)

```
print(not (True or False) and True)
```

**Output**

```
False
```

**Evaluation**

```
not (True or False) and True
= not True and True
```

```
= False and True
= False
```

**Summary:**

- `not` has the highest precedence.
- `and` has higher precedence than `or`.
- Parentheses are evaluated before logical operators.

## Example 1: Arithmetic + Relational + Logical Operators

```
x = 4
y = 2

result = x ** y + 3 * 2 > 20 and not (y == 3)
print(result)
```

**Output**

```
True
```

**Step-by-Step Evaluation**

```
x ** y + 3 * 2 > 20 and not (y == 3)
= 4 ** 2 + 3 * 2 > 20 and not (2 == 3)
= 16 + 6 > 20 and not False
= 22 > 20 and True
= True and True
= True
```

**Evaluation Order for the above expression:**

1. Arithmetic operators → `**`, `*`, `+`
2. Relational operator → `>`, `==`
3. Logical operator → `not`, `and`

---

## Example 2: Arithmetic + Relational + Logical Operators

```
a = 10
b = 5
```

```
result = a / b + 6 - 1 <= 10 or not (a < b)
print(result)
```

**Output**

```
True
```

**Step-by-Step Evaluation**

```
a / b + 6 - 1 <= 10 or not (a < b)
= 10 / 5 + 6 - 1 <= 10 or not (10 < 5)
= 2.0 + 6 - 1 <= 10 or not False
= 8.0 - 1 <= 10 or True
= 7.0 <= 10 or True
= True or True
= True
```

**Overall Operator Evaluation Order:**

1. Arithmetic Operators ( `**` , `*` , `/` , `//` , `%` , `+` , `-` )
2. Relational Operators ( `>` , `<` , `>=` , `<=` , `==` , `!=` )
3. Logical Operators ( `not` → `and` → `or` )

# 9. Type Conversion

## 9.1 Implicit Conversion (Automatic)

Python automatically converts lower data type to higher to avoid data loss. **int** → **float**

```
num_int = 6
num_float = 2.8
result = num_int + num_float   # int converted to float
print(type(result))            # O/P: 8.8 --> class 'float'
```

```
num_int = 6
num_int2 = 2
result = num_int / num_float   # int converted to float
print(type(result))            # O/P: 3.0 --> class 'float'
```

## 9.2 Explicit Conversion (Type Casting)

Explicit conversion (type casting) is performed by the programmer using Python's built-in functions.

| Function | Converts To |
|---|---|
| `int()` | Integer |
| `float()` | Floating-point number |
| `str()` | String |
| `bool()` | Boolean |
| `list()` | List |
| `tuple()` | Tuple |
| `dict()` | Dictionary |

### Example 1: int()

```
s = "125"
num = int(s) + 25
print(num)
```

**Output**

```
150
```

---

### Example 2: float()

```
pi = float("3.14")
print(pi)
```

**Output**

```
3.14
```

---

### Example 3: str()

```
n = 100
text = str(n)
print(text)
print(type(text))
```

**Output**

```
100
<class 'str'>
```

---

## Example 4: bool()

```
print(bool(0))
print(bool(25))
```

**Output**

```
False
True
```

---

## Example 5: list()

```
t = (10, 20, 30)
L = list(t)
print(L)
```

**Output**

```
[10, 20, 30]
```

---

## Example 6: tuple()

```
L = [10, 20, 30]
t = tuple(L)
print(t)
```

**Output**

```
(10, 20, 30)
```

---

### Example 7: dict()

```
pairs = [('a', 1), ('b', 2), ('c', 3)]
d = dict(pairs)
print(d)
```

**Output**

```
{'a': 1, 'b': 2, 'c': 3}
```

**Remember:**

- `int()` converts a value to an integer.
- `float()` converts a value to a floating-point number.
- `str()` converts a value to a string.
- `bool()` converts a value to `True` or `False`.
- `list()` converts an iterable (such as a tuple or string) into a list.
- `tuple()` converts an iterable into a tuple.
- `dict()` creates a dictionary from key-value pairs or another dictionary.

# 10. Input / Output from Console

## 10.1 Input Function

`input([prompt])` → always returns a **string**. Use explicit conversion for numeric data.

```
name = input("Enter student name: ")
age = int(input("Enter age: "))
```

## 10.2 Output Using print()

`print(value1, value2, ..., sep=' ', end='\n')`

- `sep` : separator between multiple values (default space)
- `end` : what to print at the end (default newline)

## 📝 String Formatting Methods

### Method 1: .format() Method (Python 2.7+ / 3.0+)

**Definition:** The `.format()` method provides a flexible way to format strings using placeholders `{}`.

- Placeholders `{}` are replaced by values passed to `.format()`.
- Can use positional arguments: `"{} {}".format("Hello", "World")`
- Can use indexed placeholders: `"{0} {1}".format("Hello", "World")`
- Can use named placeholders: `"{name} is {age} years old".format(name="Riya", age=17)`
- Supports formatting specifiers: `"{:.2f}".format(3.14159)` → "3.14"

```
# Positional arguments
print("{} scored {} marks".format("Aryan", 95))           # Aryan scored 95
marks

# Indexed placeholders
print("{0} is from {1} and {0} loves {2}".format("Riya", "Delhi", "Python"))
# Riya is from Delhi and Riya loves Python

# Named arguments
print("{name} is {age} years old".format(name="Kavya", age=17))
# Kavya is 17 years old

# Formatting specifiers
print("Pi value: {:.2f}".format(3.14159))               # Pi value: 3.14
print("Percentage: {:.1f}%".format(95.678))             # Percentage: 95.7%

# Alignment using .format()
print("{:<10}".format("left"))        # "left      " (left aligned, width 10)
print("{:^10}".format("center"))       # "  center  " (center aligned, width
10)
print("{:>10}".format("right"))        # "     right" (right aligned, width
10)

# Alignment with fill character
print("{:*<10}".format("left"))        # "left******"
print("{:*^10}".format("center"))       # "**center**"
print("{:*>10}".format("right"))        # "*****right"

# Alignment with numbers
print("{:<10}".format(123))            # "123       "
print("{:^10}".format(456))             # "   456    "
print("{:>10}".format(789))             # "       789"
```

```
# Combined alignment with formatting
print("{:<10}{:>10}".format("Name", "Marks"))    # "Name          Marks"
print("{:<10}{:>10.2f}".format("Riya", 95.5))    # "Riya           95.50"
```

## Method 2: f-strings (Formatted String Literals) - Python 3.6+

**Definition:** f-strings provide a concise and readable way to embed expressions inside string literals. Prefix the string with `f` or `F` and use curly braces `{}` to include variables or expressions.

- Expressions inside `{}` are evaluated at runtime and converted to strings.
- More readable and faster than `.format()` or concatenation.
- Supports all formatting specifiers like `{variable:.2f}`.
- Can include any valid Python expression (arithmetic, function calls, etc.).

```
name = "Riya"
age = 17
marks = 95.5

# Using f-string
print(f"Student: {name}, Age: {age}, Marks: {marks}")           # Student:
Riya, Age: 17, Marks: 95.5
print(f"Hello {name}! You scored {marks:.1f}%")                 # Hello Riya!
You scored 95.5%
print(f"Next year you will be {age + 1} years old")             # Next year
you will be 18 years old

# Alignment using f-strings
print(f"{'left':<10}")        # "left      " (left aligned, width 10)
print(f"{'center':^10}")       # "  center  " (center aligned, width 10)
print(f"{'right':>10}")        # "     right" (right aligned, width 10)

# Alignment with fill character
print(f"{'left':*<10}")        # "left******"
print(f"{'center':*^10}")       # "**center**"
print(f"{'right':*>10}")        # "*****right"

# Alignment with numbers
print(f"{123:<10}")            # "123       "
print(f"{456:^10}")             # "   456    "
print(f"{789:>10}")             # "       789"

# Alignment with variables
print(f"{name:<10}{marks:>10.2f}")    # "Riya           95.50"
print(f"{'Name':<10}{'Marks':>10}")   # "Name         Marks"

# Combined alignment with expressions
```

```
print(f"{name.upper():*^20}")          # "*******RIYA*******"

# Traditional print() with sep and end
print("Class", "12", sep="-", end=" * ")                        # Class-12 *
print("CBSE")                                                   # CBSE
(continues on same line due to end=" * ")
```

✅ **Alignment Specifiers Summary:**

- `<` → Left align (default for strings)
- `>` → Right align (default for numbers)
- `^` → Center align
- `:` followed by fill character (e.g., `*`, `-`, `_`)
- Syntax: `{value:fill_character alignment width}`
- Examples: `{:<10}`, `{:*^20}`, `{:>15}`

✅ **Comparison: .format() vs f-strings**

- **Readability:** f-strings are more readable as variables are directly embedded.
- **Performance:** f-strings are faster than `.format()`.
- **Flexibility:** `.format()` works in older Python versions (2.7+), f-strings require Python 3.6+.
- **Expressions:** Both support expressions, but f-strings are more intuitive.
- **Best Practice:** Use f-strings for Python 3.6+ projects, .format() for backward compatibility.

## 11. Use of Comments

Comments are ignored by interpreter; used for documentation, readability, and debugging.

- **Single-line comment:** starts with `#`
- **Multi-line comment:** using triple quotes `''' ... '''` or `""" ... """` (docstring style, but works as comment)

```
# This program calculates average marks
marks1 = 89  # store subject marks
"""
   Following block prints the total
   using arithmetic addition
"""
total = marks1 + 78
```

## 📌 Quick Revision Table

| Topic | Key Point / Syntax Example |
|---|---|
| Character set | Unicode letters, digits, symbols, whitespace |
| Tokens | Keyword, identifier, literal, operator, punctuator |
| L-value | Left side of = (memory location) |
| R-value | Right side = constant/expression value |
| Identity operators | `is` , `is not` → object identity |
| Membership operators | `in` , `not in` → sequence membership |
| Precedence (high to low) | () → ** → +x, -x → *,/,//,% → +,- → comparison → not → and → or |
| Implicit conversion | Automatic e.g., int→float |
| Explicit conversion | int("45"), float("67.8") |
| input() | Always returns string, convert if needed |
| print() | sep, end parameters |
| Comment | # single line, """ multi-line """ |

## 💻 Comprehensive Examples

```
# Demonstration of all operators & conversion
x = 10
y = 3
print("Arithmetic:", x + y, x % y, x**y)
print("Relational:", x > y, x == 10)
print("Logical:", (x > 5) and (y < 5))
# augmented
x += 5   # x becomes 15
print("Augmented x:", x)
# identity
lst1 = [1,2]
lst2 = lst1
print(lst1 is lst2)   # True
# membership
print(2 in [1,2,3])   # True
# type conversion explicit
```

```
s = "100"
num = int(s) + 50
print("Explicit conversion result:", num)

# Input-Output demo in concept (simulated)
# name = input("Enter name: ")
# print(f"Hello {name}")
```

**Exam Tips:**

- Difference between `=` and `==` – assignment vs equality.
- `is` vs `==` : identity vs value equality.
- Precedence based evaluation questions are common in CBSE practicals & theory.
- Augmented assignment saves code length.

---